[COLLEGE / UNIVERSITY NAME]

Department of Computer Engineering

Operating Systems – Course Project

REVIEW 1

Problem, Literature and Proposed Idea

(Conference Paper Track)

MediQueue: An AI-Assisted Emergency Department Queue Built on Priority Scheduling and Process Synchronization

Submitted by

Name	Register Number
[Student Name 1]	[Reg. No.]
[Student Name 2]	[Reg. No.]
[Student Name 3]	[Reg. No.]

Under the guidance of

[Guide Name], [Designation]

Department of Computer Engineering

1. Project Title

MediQueue: An AI-Assisted Emergency Department Queue Built on Priority Scheduling and Process Synchronization

The project is a web application for the casualty / emergency department of a hospital. It decides which waiting patient a doctor should see next, makes sure the queue stays correct when many staff use it together, and uses machine learning to help the triage nurse and to tell patients roughly how long they have to wait.

2. Problem Statement

In many small and medium hospitals the emergency waiting area still works on a token or a register. Patients are called mostly in the order they came in. So a person with chest pain and low BP can end up sitting behind a few people with minor cuts and sprains. Some hospitals do triage, but the triage colour is usually written on a slip and the doctor or ward boy still calls patients in whatever order seems right at that moment.

There are two more problems. Patients are never told how long they will have to wait, so many of them keep asking at the desk or leave without being seen. And when three or four doctors and a couple of nurses work on the same queue, it happens that two doctors call the same patient, or the same observation bed is promised to two patients.

Problem statement: To build a system that calls emergency patients in order of medical urgency without making low-priority patients wait forever, that works correctly when several staff members use it at the same time, and that gives the triage nurse an explainable suggestion and the patients a realistic waiting time.

3. Motivation

When we studied CPU scheduling in our OS course, we noticed that a hospital casualty ward behaves almost exactly like a scheduler. The doctors are like processors, the waiting room is the ready queue, every patient has a priority, and nobody knows in advance how long a consultation will take (the burst time). Starvation also happens in real life: a patient with a minor problem can keep getting pushed back for hours.

The concurrency part is also real. Doctors in different rooms press "next patient" at the same time and nurses admit patients from different computers, so the same race

conditions we studied in synchronization can actually happen here. We wanted a project where the OS concepts are needed for a real reason and are not just added to show them.

Other reasons for choosing this topic:

- In emergency care a delay of even 10 to 15 minutes matters. The ESI triage guidelines expect the most urgent patients to be seen immediately [1].
- Not knowing the waiting time is one of the biggest complaints of patients and their relatives.
- Most existing solutions are expensive hospital software. We wanted something that can run on a normal laptop without a separate database server.

4. Objectives

1. To design a priority based dispatcher with one level for each triage (ESI) category, which works with more than one doctor at a time and uses aging so that low-priority patients do not starve.
2. To allow preemption only for resuscitation (ESI 1) patients, since stopping a doctor in the middle of a normal consultation is not practical.
3. To make "call next patient" and bed allocation safe when used concurrently, by treating them as critical sections and modelling the observation beds as a counting semaphore.
4. To train a machine learning model that suggests the triage level from vital signs and shows the reason, while the nurse takes the final decision.
5. To predict consultation time and use it to estimate and display the waiting time for each patient.
6. To build a working web application with separate screens for the nurse, doctor, waiting room and admin.
7. To compare our scheduling method with FCFS, SJF and normal priority scheduling using simulation.

5. Existing System / Existing Approaches

5.1 Token / register system

The patient takes a token and doctors call tokens one by one. It is cheap and easy, but there is no idea of urgency at all. A serious patient gets seen early only if some staff member notices and moves them ahead.

5.2 Manual triage with colour codes

A trained nurse gives each patient a level (red, orange, yellow, green, blue) using a protocol like ESI or the Manchester Triage System. This is medically correct, but deciding who goes in next is still done by hand, a green patient can wait for hours, and the data is not stored anywhere for analysis.

5.3 Hospital information system (HIS) triage module

Bigger hospitals enter the triage level into their HIS. Mostly these modules just store the level like a form. They do not decide the calling order, do not estimate waiting time, and the rules are fixed.

5.4 Early warning scores (NEWS2)

NEWS2 gives points for abnormal vital signs [2]. It is simple and has been validated for finding patients who are getting worse [10], but it does not look at the complaint or pain. For example a stroke patient with normal vitals can get a very low score.

Table 1: Comparison of existing approaches

Feature	**Token system**	**Manual triage**	**HIS module**	**NEWS2**
Calling order based on urgency	No	Manual	Partly	No
Prevents long waiting (starvation)	No	No	No	No
Safe when many staff use it	No	No	Yes	–
Triage suggestion	No	Protocol	Fixed rules	Score
Reason shown for the level	No	No	No	Points
Waiting time shown to patient	No	No	No	No
Queue for observation beds	No	Manual	Partly	No

6. Literature Survey

The papers we referred to are listed in Table 2. They are from three areas: OS scheduling and synchronization, triage and machine learning, and emergency department patient flow / waiting time.

Table 2: Literature survey

S.No	Title / Authors / Year	Objective	Method used	Dataset	Findings	Limitations
1	Operating System Concepts – Silberschatz, Galvin, Gagne (2018) [3]	Covers CPU scheduling and process synchronization	FCFS, SJF, priority, RR, MLFQ, aging, semaphores	Textbook	Priority scheduling can starve low priority jobs; aging solves it	Written for CPUs; aging has no upper limit, which is unsafe for patients
2	Cooperating Sequential Processes – E. W. Dijkstra (1965) [4]	Mutual exclusion between processes	Semaphores with wait (P) and signal (V)	–	Counting semaphore can manage a pool of identical resources	Theory only; has to be mapped to database transactions in a web app
3	Queueing Systems Vol. II – L. Kleinrock (1976) [5]	Study of priority queues	Queueing theory	Analytical	With strict priority, low priority waiting time grows very fast as load increases	Assumes known service time distribution
4	Emergency Severity Index Handbook v4 – Gilboy et al. (2012) [1]	5-level triage for EDs	Decision points: life saving need, high risk, resources, vitals	Guideline	Reliable and widely used	Gives only the level, not the calling order or waiting time
5	NEWS2 – Royal College of Physicians (2017) [2]	Early warning score	Points for RR, SpO2, BP, HR, consciousness, temperature	National guideline	Simple and transparent	Ignores complaint and pain
6	Ability of NEWS to discriminate patients at risk – Smith et al. (2013) [10]	Validate NEWS	AUROC analysis	Hospital vital sign records	NEWS identifies high risk patients well	Ward patients, not ED triage
7	ML-based electronic triage – Levin et al. (2018) [6]	Improve ED triage	Random forest	Large ED visit records (USA)	Separated patients by outcome better than ESI	Needs big private data; no scheduling or waiting time
8	Predicting hospital admission at ED triage – Hong et al. (2018) [7]	Predict admission at triage	Logistic regression, XGBoost, deep network	About 5.6 lakh ED visits	AUC around 0.92 with full data	Predicts admission, not urgency; not linked to queue
9	ED triage prediction using	Compare ML with	Lasso, random forest,	US national ED	ML predicted critical care	Retrospective only

	ML – Raita et al. (2019) [8]	conventional triage	gradient boosting, DNN	survey (NHAMCS)	and hospitalisation better	
10	Accurate ED wait time prediction – Ang et al. (2016) [11]	Predict waiting time	Q-Lasso (queue features + Lasso)	Data from US hospitals	Much lower error than rolling average	Does not decide the service order
11	Complexity-augmented triage – Saghafian et al. (2014) [12]	Use complexity in prioritising	Analytical model + simulation	Simulated ED	Using treatment length with urgency improves flow	Policy study, no software
12	OR/MS contributions to ED patient flow – Saghafian et al. (2015) [13]	Review	Literature review	–	Prioritisation rules strongly affect waiting and safety	Few methods are used in real hospitals
13	LIME: Why should I trust you? – Ribeiro et al. (2016) [14]	Explain ML predictions	Local perturbation	Benchmarks	Users trust models more when reasons are shown	General method; we use a simpler what-if version
14	MIMIC-IV-ED – Johnson et al. (2023) [9]	Public ED dataset	De-identified ED stays	About 4 lakh ED stays	Has triage vitals and acuity	Needs credentialed access

7. Research Gap

After going through the papers, we found these gaps:

- The ML triage papers [6]–[8] only predict the urgency or the outcome. None of them uses this prediction to actually decide which patient the doctor should call next.
- Aging in textbooks [3] keeps increasing the priority without any limit. In a hospital this means a minor case that has waited long could go ahead of a heart attack patient. We did not find any work that limits aging for this reason.
- Waiting time prediction [11] is done separately from the scheduling rule. It is not based on the actual order in which patients will be called.
- None of the papers talks about what happens when many staff use the queue at the same time, even though that is where double calling and double bed allocation come from.
- Most ML models are black boxes and the nurse cannot see why a level was given, and there is no record of when the nurse disagreed.
- Almost all studies use large US hospital data and assume good IT setup. Very little is aimed at small hospitals in India.

8. Proposed Solution

We propose MediQueue, a web based queue and triage system that treats the emergency department like a small operating system. Instead of using many OS concepts, we use only the two that the hospital actually needs.

First, priority scheduling with aging. When a doctor presses "Call next patient", the system picks the patient with the most urgent level, and if two patients have the same level, the one who came first. Every 30 minutes of waiting moves a patient up by one level, but only up to level 3. Levels 1 and 2 are kept only for really serious patients. A doctor is asked to stop a consultation only when an ESI 1 patient comes and all doctors are busy.

Second, process synchronization. Calling the next patient, admitting a patient to a bed and marking a bed as cleaned are done inside critical sections using the database lock. The observation beds work like a counting semaphore: admitting a patient is wait(), a cleaned bed is signal(), and patients waiting for a bed form the blocked queue.

Along with this there are three AI parts:

- A random forest model that suggests the triage level from the vital signs, pain, consciousness and complaint. It shows which values increased the urgency. Some danger signs (like SpO2 below 85) directly give ESI 1. The nurse can change the level but has to write a reason.
- A second model that predicts how long the consultation will take.
- A waiting time estimate. We first run the scheduler forward using the predicted times and then correct it with a gradient boosting model. The waiting room screen shows a range like 20–35 min.

9. Proposed Architecture / Block Diagram

The system has three tiers as shown in Fig 1. The screens for nurse, doctor, beds, waiting room and admin refresh every 5 seconds from the server. The Flask server handles each request in a separate thread, which is why the dispatcher and the bed module need synchronization. SQLite stores the data and the trained models are loaded from files.

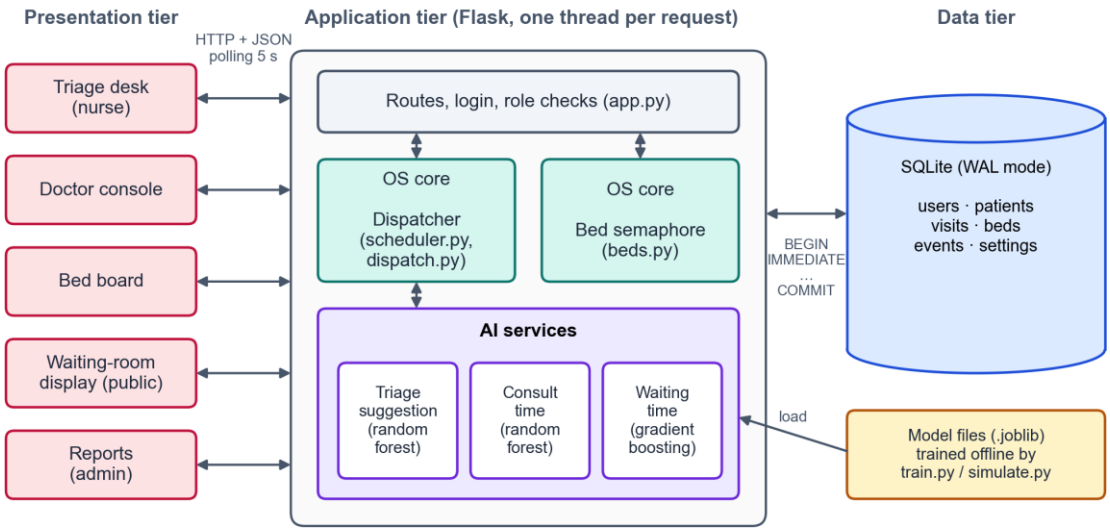


Fig 1: Block diagram / architecture of MediQueue

Fig 2 shows the context level DFD with the four users of the system.

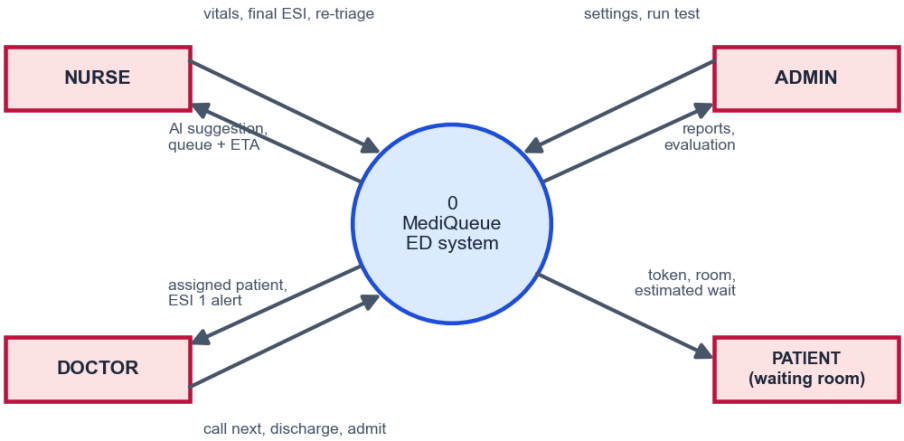


Fig 2: Context level (Level 0) DFD

10. Proposed Methodology

1. Study the ESI and NEWS2 guidelines and list the decisions taken at the triage desk, in the doctor rooms and in the observation ward.
2. Prepare the dataset (explained in section 12).
3. Train the triage model and the consultation time model, check them with 5-fold cross validation and set the safety margin to reduce under-triage.
4. Write the scheduling logic as separate functions so that the same code is used in the website and in the simulator.
5. Put the dispatch and bed operations inside database transactions (BEGIN IMMEDIATE ... COMMIT).
6. Develop the web application using Flask, SQLite, HTML, CSS and JavaScript.
7. Simulate many 12 hour shifts and compare FCFS, SJF, priority and our method. Also try different aging intervals.
8. Test all the flows and run a stress test with many threads, with and without locking.

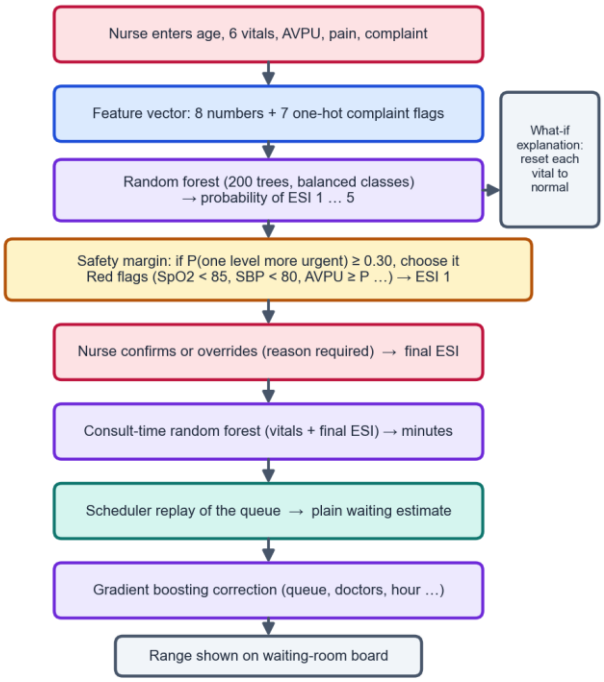


Fig 3: Flow of the AI part

11. Initial Algorithm / Workflow

11.1 Call next patient

```
CALL_NEXT(doctor):
  BEGIN IMMEDIATE            // lock - start of critical section
  if doctor already has a patient: ROLLBACK and stop
  W = all patients with status 'waiting'
  for each p in W:
      if p.esi <= 2: level = p.esi
      else: level = max(3, p.esi - floor(wait_time(p) / 30))
  choose p with smallest (level, arrival time)
  UPDATE status = 'in_consult' WHERE id = p AND status = 'waiting'
  if no row updated: ROLLBACK and stop
  COMMIT                     // unlock - end of critical section
```

11.2 Preemption for ESI 1

```
if an ESI 1 patient is waiting and all doctors are busy:
    d = doctor with the least urgent patient (ESI 3 or more)
    if d found: ask d to pause current patient and see ESI 1 patient
    else: show message to call another doctor
```

11.3 Bed allocation (counting semaphore)

```
wait(beds):                          signal(beds):   // bed cleaned
  if a bed is free:                    if someone is waiting for a bed:
      give bed to patient                  give bed to most urgent one
  else:                                  else:
```

```
patient status = boarding        bed status = free
```

11.4 Triage suggestion

```
x = [age, HR, BP, RR, SpO2, temp, AVPU, pain, complaint]
P = random_forest.predict_proba(x)
level = most probable level
if P(one level more urgent) >= 0.30: level = level - 1
if any danger sign: level = 1
show level and reasons, nurse confirms or changes it
```

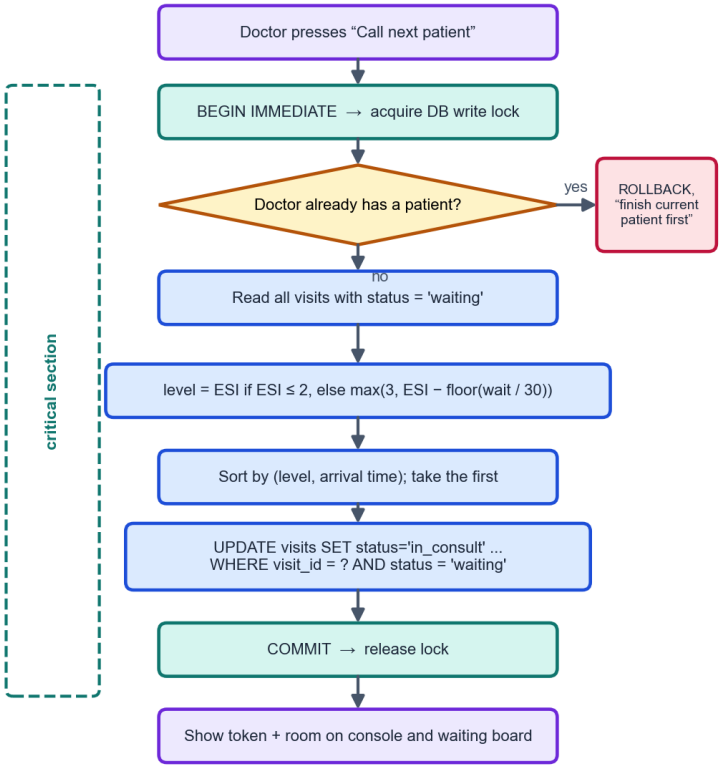


Fig 4: Workflow of call next patient

12. Dataset / Source of Data

We could not get real hospital triage data. The public dataset MIMIC-IV-ED [9] needs credentialed access and an ethics course, which we could not complete in this semester. So for now we are using a synthetic dataset.

- 3000 patients generated from 7 types of cases: minor injury, fever, abdominal pain, trauma, breathing difficulty, chest pain and stroke. Age and vital signs are taken from normal ranges for each type.

- The triage level is given using NEWS2 [2] and the ESI rules [1]. About 6% of the levels are changed by one to copy the way different nurses can disagree.
- The consultation time is generated based on the complaint, the level and the age.
- For testing the scheduler we simulate 12 hour shifts (8 am to 8 pm) with 4 doctors and more patients in the late morning and evening.
- Later we plan to use MIMIC-IV-ED, and in a real hospital the system's own records can be used to retrain the models.

Table 3: Input features used

Feature	**Range / type**	**Collected from**
Age	0 – 110 years	Patient
Heart rate	beats per minute	Monitor
Systolic BP	mmHg	BP machine
Respiratory rate	breaths per minute	Nurse
SpO2	50 – 100 %	Pulse oximeter
Temperature	°C	Thermometer
Consciousness	AVPU scale (0 – 3)	Nurse
Pain score	0 – 10	Patient
Complaint	7 categories	Nurse

13. Evaluation Parameters

Table 4: Evaluation parameters

Part	**Parameter**	**Reason**
Triage model	Accuracy, F1 score, confusion matrix	Overall correctness
Triage model	Under-triage % (less urgent than actual)	Most dangerous mistake
Triage model	Over-triage %, accuracy within one level	Cost of being careful
Triage model	Comparison with NEWS2 only	To see if ML helps
Consultation time	Mean absolute error in minutes	How good the burst time guess is
Scheduler	Average wait of ESI 1–2 patients	Speed for serious patients
Scheduler	% of patients seen within target time	Following ESI guidelines
Scheduler	95th percentile wait of ESI 4–5, worst wait	Checking starvation
Scheduler	Average wait of all patients, preemptions	Overall efficiency
Waiting time	Mean absolute error in minutes	Accuracy of the display
Synchronization	Number of double assignments	Correctness
Synchronization	Time per request with and without lock	Cost of locking

14. Expected Outcomes

- A working web application for the emergency department with triage desk, doctor screen, bed board, waiting room display and admin reports.
- Serious patients (ESI 1–2) should be seen much faster than with the token system, without low priority patients waiting for many hours like in plain priority scheduling.
- Under-triage should be low (around 5%) and the accuracy should be clearly better than using NEWS2 alone.
- The waiting time shown to patients should be closer to the real waiting time than a simple queue based estimate.
- No patient should be given to two doctors and no bed to two patients, even when many people use the system at the same time.
- A conference paper explaining the design, results and limitations.

15. References

[1] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4, Implementation Handbook. Rockville, MD: AHRQ, Publication No. 12-0014, 2012.

[2] Royal College of Physicians, National Early Warning Score (NEWS) 2: Standardising the Assessment of Acute-Illness Severity in the NHS. London: RCP, 2017.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[4] E. W. Dijkstra, "Cooperating sequential processes," Technological University, Eindhoven, Tech. Rep. EWD-123, 1965.

[5] L. Kleinrock, Queueing Systems, Volume II: Computer Applications. New York: Wiley, 1976.

[6] S. Levin et al., "Machine-learning-based electronic triage more accurately differentiates patients with respect to clinical outcomes compared with the Emergency Severity Index," Ann. Emerg. Med., vol. 71, no. 5, pp. 565-574, 2018.

[7] W. S. Hong, A. D. Haimovich, and R. A. Taylor, "Predicting hospital admission at emergency department triage using machine learning," PLoS ONE, vol. 13, no. 7, e0201016, 2018.

[8] Y. Raita, T. Goto, M. K. Faridi, D. F. M. Brown, C. A. Camargo, and K. Hasegawa, "Emergency department triage prediction of clinical outcomes using machine learning models," Crit. Care, vol. 23, art. 64, 2019.

[9] A. Johnson, L. Bulgarelli, T. Pollard, L. A. Celi, R. Mark, and S. Horng, "MIMIC-IV-ED (version 2.2)," PhysioNet, 2023.

[10] G. B. Smith, D. R. Prytherch, P. Meredith, P. E. Schmidt, and P. I. Featherstone, "The ability of the National Early Warning Score (NEWS) to discriminate patients at risk of early cardiac arrest, unanticipated intensive care unit admission, and death," Resuscitation, vol. 84, no. 4, pp. 465-470, 2013.

[11] E. Ang, S. Kwasnick, M. Bayati, E. L. Plambeck, and M. Aratow, "Accurate emergency department wait time prediction," Manuf. Serv. Oper. Manag., vol. 18, no. 1, pp. 141-156, 2016.

[12] S. Saghafian, W. J. Hopp, M. P. Van Oyen, J. S. Desmond, and S. L. Kronick, "Complexity-augmented triage: A tool for improving patient safety and operational efficiency," Manuf. Serv. Oper. Manag., vol. 16, no. 3, pp. 329-345, 2014.

[13] S. Saghafian, G. Austin, and S. J. Traub, "Operations research/management contributions to emergency department patient flow optimization: Review and research prospects," IIE Trans. Healthc. Syst. Eng., vol. 5, no. 2, pp. 101-123, 2015.

[14] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135-1144.

[15] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[16] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, 2001.

[17] SQLite Consortium, "File locking and concurrency in SQLite version 3." [Online]. Available: https://www.sqlite.org/lockingv3.html